

QA Basics Notes and Test Scenarios

Introduction to QA

Quality Assurance (QA) is the process of checking whether a software application works correctly and meets the expected requirements before it is released to users. It is a systematic approach designed to ensure that products and services consistently meet customer expectations, achieve service targets, and adhere to industry standards. QA helps improve software quality, reduce bugs, and provide a better user experience.

Goals of QA

- Find bugs before users find them.
- Improve software quality.
- Ensure application stability and security.
- Verify requirements are working correctly.
- Increase customer satisfaction.

SDLC (Software Development Life Cycle) The Software Development Life Cycle (SDLC) is a well-structured process that guides software development projects from start to finish. It provides a clear framework for planning, building, and maintaining software, ensuring that development is systematic and meets quality standards. SDLC is the process of planning, writing, modifying, and maintaining software. SDLC is the process used to develop software step by step.

Phases of SDLC

1. Requirement Gathering
2. Planning
3. Design
4. Development
5. Testing
6. Deployment
7. Maintenance

STLC (Software Testing Life Cycle)

The Software Testing Life Cycle (STLC) is a systematic sequence of phases executed by Quality Assurance (QA) teams to validate software quality. STLC is the testing process followed by QA teams. It ensures the application meets requirements and detects defects early, consisting of Requirement Analysis, Test Planning, Test Design, Environment Setup, Execution, etc.

Phases of STLC

1. Requirement Analysis
2. Test Planning
3. Test Case Creation
4. Test Environment Setup
5. Test Execution
6. Bug Reporting
7. Test Closure

Types of Testing

1. Functional Testing

Checks whether the application features work according to requirements.

Example:

- Login button works correctly
- User can register successfully

2. Smoke Testing

Basic testing done to check whether the application is stable for further testing.

Example:

- App opens successfully
- Main pages load correctly

3. Sanity Testing

Checks whether a specific feature or bug fix works correctly after small changes.

Example:

- Verify password reset works after bug fix

4. Regression Testing

Ensures old features still work after adding new updates.

Example:

- Login still works after payment feature update

5. Negative Testing

Checks how the application behaves with invalid input.

Example:

- Enter wrong password during login
- Submit empty registration form

6. UI Testing

Checks user interface elements like buttons, fonts, colors, and alignment.

7. Compatibility Testing

Checks whether the application works on different devices, browsers, and operating systems.

Bug Life Cycle

The Bug Life Cycle (Defect Life Cycle) describes the journey of a defect from the moment it is identified until it is resolved and verified. Understanding this process helps QA teams, developers, and project managers track issues effectively and maintain software quality.

1. New

A tester discovers a defect and reports it in a bug tracking tool such as Jira. The bug is waiting for review.

2. Assigned

The bug is assigned to a developer or development team for investigation and resolution.

3. Open

The developer reviews the defect, confirms it is valid, and begins working on a fix.

4. Fixed

The developer implements the solution and marks the issue as fixed.

5. Retest

The QA tester verifies whether the fix works correctly and ensures the original issue no longer occurs.

6. Closed

If the fix is successful, the defect is closed and considered resolved.

Severity vs Priority

Severity and Priority are two of the most important concepts in software testing. Although they are related, they measure different aspects of a defect.

Severity

Severity indicates how much a defect impacts the application's functionality, performance, or stability. It is usually determined by the QA tester.

Examples:

- Application crashes during payment processing.
- Users cannot log in to their accounts.
- Data is lost after saving.

Priority

Priority indicates how urgently a defect should be fixed based on business needs, customer impact, and release deadlines. It is often decided by the Product Manager, Project Manager, or Team Lead.

Examples:

- Incorrect company logo on the homepage before a major product launch.
- Typographical error in a marketing banner.
- Broken feature affecting a large number of customers.

Common Severity and Priority Combinations

High Severity + High Priority

Payment page crashes when users submit transactions.

High Severity + Low Priority

A rarely used admin feature fails, but it does not affect customers immediately.

Low Severity + High Priority

A spelling mistake appears on the homepage during a major marketing campaign.

Low Severity + Low Priority

Minor UI alignment issue on an internal settings page.

Components of a Test Case

Field	Description
Test Case ID	Unique identifier
Test Scenario	What needs to be test
Preconditions	Conditions before testing
Test Steps	Actions performed
Test Data	Input values used
Expected Result	Expected output
Actual Result	Actual application behavior
Status	Pass or Fail

Ensures old features still work after adding new updates.

Example:

- Login still works after payment feature update

5. Negative Testing

Checks how the application behaves with invalid input.

Example:

- Enter wrong password during login
- Submit empty registration form

6. UI Testing

Checks user interface elements like buttons, fonts, colors, and alignment.

7. Compatibility Testing

Checks whether the application works on different devices, browsers, and operating systems.